

Design Document

1. System Model and Assumptions

Our System consists of two components: (1) an AFS-inspired distributed filesystem for storing input datasets and output results across three replica servers, and (2) a coordinator-worker application that reads input files from AFS, tests every number for primality in parallel, deduplicates results, and writes the final output back to AFS. Both components are implemented from scratch in Go using gRPC.

Assumptions

- **Crash-stop fault model:** Processes fail by crashing and do not exhibit Byzantine behavior. A crashed process either restarts cleanly or stays down permanently.
- **Reliable network:** Messages between processes are not corrupted, duplicated, or reordered by the network. Connections may time out if a process has crashed.
- **Static input files:** Input datasets are pre loaded on all servers and never modified. Only output files (primes.txt, snapshots) are written during execution.
- **$f = 1$ tolerance:** With 3 replica servers, the system tolerates one server failure ($2f+1 = 3$ for $f = 1$). The system does not handle simultaneous failure of 2+ servers.
- **Synchronous RPCs:** All coordinator worker communication uses synchronous gRPC streaming. The coordinator waits for the full response before proceeding to the next operation.

2. Architecture Overview

Component	Role	Key Mechanism
AFS Server (3 replicas)	Persistent storage for input and output files	Primary backup replication, bully leader election, SyncState recovery
AFS Client Library	File API for applications (Open/Read/Write/Close)	Whole file caching, TestAuth version check, retry with failover
Coordinator	Manage multiple workers together to find prime number	File discovery, round-robin assignment, cross-file dedup, Chandy Lamport snapshots
Worker (N instances)	Processes one input file at a time	AFS client, Miller-Rabin primality, streaming results in 10K batches

3. AFS Replication: Primary Backup with Synchronous Writes

Why primary backup instead of Raft/Paxos: The workload is read heavy with occasional writes (output files, snapshots). Input files are static. With only 3 servers, the overhead of a full consensus protocol (log replication, term management, log compaction) is unnecessary. Primary-backup with synchronous parallel replication provides the same consistency guarantee for this workload at a fraction of the complexity.

Write path: The client sends all mutating RPCs (StoreFile, CreateFile, DeleteFile) to the primary. The primary writes to local disk using atomic write-rename (write to .tmp, then `os.Rename`), bumps the version counter, and replicates to all backups in parallel using `sync.WaitGroup`. The primary waits for all backup ACKs before responding to the client. This guarantees that when a write is acknowledged, all live servers have the data.

Idempotency: Every mutating RPC carries (clientID, reqSeq). The server maintains a dedup table. If the same key arrives again (client retry after timeout), the server returns the cached response without re-executing. This provides exactly-once semantics.

4. Leader Election: Bully Algorithm (Lowest-ID)

Backups monitor heartbeats from the primary (1s interval, 3s timeout, 6s startup grace). When a backup detects no heartbeat for 3 seconds, it triggers an election: it pings all peers via `GetPrimary`, determines which are alive, and selects the server with the lowest lexicographic ID as the winner. Since IDs are deterministic strings (`s1`, `s2`, `s3`), all live servers independently compute the same winner without coordination messages. The winner promotes itself and broadcasts `AnnouncePrimary` to all peers. Backups then run `SyncState` to pull any files they missed. On restart, a server calls `findExistingPrimary` to prevent split-brain: if another primary already exists, it starts as a backup instead.

5. Chandy-Lamport Consistent Snapshots

The prime-finder application uses the Chandy-Lamport algorithm for consistent global snapshots, adapted for the coordinator-worker topology. Snapshots enable crash recovery without restarting computation from scratch.

Protocol Steps

- **Initiate (coordinator):** After each file completes, the coordinator records its local state: completed files, pending files, and all collected primes so far.
- **Send markers:** The coordinator sends an HTTP POST marker to every live worker's `/snapshot` endpoint. This is the Chandy Lamport marker message on each outgoing channel.
- **Record worker state:** Upon receiving the marker, each worker records its local state (completed files, primes found) and responds with its snapshot as an ACK.
- **Record channel state:** Because `ProcessFile` is a synchronous streaming RPC, the coordinator only initiates a snapshot after collecting the full result. Therefore, all coordinator-worker channels are empty at snapshot time. The channel state is recorded explicitly as `{messages_in_flight: 0}` for each channel.
- **Persist:** Both coordinator and worker snapshots are written to AFS as JSON files (`snapshot_coordinator.json`, `snapshot_w1.json`, etc.), automatically replicated to all 3 servers.
- **Recovery:** On restart, the coordinator loads its snapshot from AFS, skips completed files, reuses collected primes, and resumes processing only the pending files.

Why channels are trivially empty: In standard Chandy-Lamport, the initiator must record messages received on each incoming channel between its local snapshot and the arrival of the marker on that channel. In our system, the coordinator calls `ProcessFile` (synchronous streaming RPC) and blocks until the worker sends all results and EOF. Only then does the coordinator initiate the snapshot. Since no RPC is in progress at snapshot time, no messages are in flight on any channel. This simplification is valid because the coordinator is both the snapshot initiator and the only process that sends/receives on all channels.

6. Distributed Prime Finding Application

Architecture: Coordinator-worker model. The coordinator discovers input files on AFS, assigns them round-robin to available workers, and dispatches each worker in its own goroutine. Workers are stateless compute nodes: they connect to AFS as clients, fetch their assigned file, test every number for primality, and stream results back in batches of 10,000 primes per gRPC message. The coordinator maintains a global dedup map to eliminate duplicates across files. After all files complete, it writes `primes.txt` to AFS.

Primality testing: We use the Miller-Rabin primality test with 12 fixed witnesses {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}.

Worker failover: If a worker fails mid processing, the coordinator's streaming Recv() returns an error. processOneFile then iterates through all other live workers and reassigns the failed file. The output remains correct because deduplication happens at the coordinator.

7. Fault Tolerance Summary

Fault	Detection	Recovery
AFS primary crash	Heartbeat timeout (3s)	Bully election, client failover, SyncState on recovery
AFS backup crash	Replication timeout	Primary continues; backup syncs on restart
Client crash before Close	N/A (local writes only)	Server never sees partial data safe by design
RPC failure (network)	Transport error	retryWithFailover (8 retries) with reconnect to new primary
Worker crash	ProcessFile stream error	Coordinator reassigns file to surviving worker
Coordinator crash	Process exit	Restart loads Chandy Lamport snapshot from AFS, resumes
Duplicate RPCs	Dedup table lookup	Cached response returned (exactly-once semantics)

8. Design Principle

Every design decision follows one principle: simplest correct design for this specific workload. Input files are static (whole-file caching always valid, no invalidation). Output is write-once (no concurrent write conflicts, primary-backup sufficient without Raft). Lab scale with 3 servers (lowest-ID election, no voting rounds needed). These properties justify every simplification while maintaining correctness and fault tolerance.